



PDF Download
3274895.3274913.pdf
14 January 2026
Total Citations: 2
Total Downloads: 174

Latest updates: <https://dl.acm.org/doi/10.1145/3274895.3274913>

RESEARCH-ARTICLE

Stepping stone graph for public movement analysis

SAMEERA KANNANGARA, University of Melbourne, Melbourne, VIC, Australia

EGEMEN TANIN, University of Melbourne, Melbourne, VIC, Australia

AARON HARWOOD, University of Melbourne, Melbourne, VIC, Australia

SHANIKA KARUNASEKERA, University of Melbourne, Melbourne, VIC, Australia

Open Access Support provided by:

University of Melbourne

Published: 06 November 2018

[Citation in BibTeX format](#)

SIGSPATIAL '18: 26th ACM SIGSPATIAL
International Conference on Advances in
Geographic Information Systems
November 6 - 9, 2018
Washington, Seattle

Conference Sponsors:
SIGSPATIAL

Stepping Stone Graph for Public Movement Analysis

Sameera Kannangara, Egemen Tanin, Aaron Harwood and Shanika Karunasekera

kannangarad@student.unimelb.edu.au, {etanin, aharwood, karus}@unimelb.edu.au

School of Computing and Information Systems

The University of Melbourne

ABSTRACT

There are many real world applications that require to identify movement of users such as identifying movement corridors, most popular paths, and nearest neighbours. If one is not given trajectories mapping to movement of people but rather sporadic location data, such as location based social network data, finding movement related information becomes difficult. Rather than processing all points in a data set given a query, a clever approach is to construct a graph, based on user locations, and query this graph for all queries. One example is the shortest path graph. However the shortest path graph can be inefficient and ineffective analysing movement, as it calculates the graph considering all points in a data set. We propose the stepping stone graph, which calculates graph considering point pairs rather than all points, that focuses on local possible movement, making it both efficient and effective for location based social network related queries. We demonstrate its uses by applying it in the aforementioned domain and comparing with the shortest path graph. We also compare its properties to a range of other graphs.

CCS CONCEPTS

• Information systems → Geographic information systems;

KEYWORDS

moving objects, shortest path, graphs

ACM Reference Format:

Sameera Kannangara, Egemen Tanin, Aaron Harwood and Shanika Karunasekera. 2018. Stepping Stone Graph for Public Movement Analysis. In *26th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (SIGSPATIAL '18)*, November 6–9, 2018, Seattle, WA, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3274895.3274913>

1 INTRODUCTION

Location based social network (LBSN) data typically has voluntarily generated content inspired by the users' surroundings along with the locality where the content is generated. Integration of social network analysis with geography is an important research area [15]. LBSN data provides a valuable source for establishing situational awareness during interesting events [11]. Proposing methods to analyze public movement using LBSN data is difficult since different

users have different posting patterns [3]. A common problem is that most users do not post content frequently enough to deduce any trajectory information. As there is an increased interest in mining movement from spatial data [6], we propose to combine location information from different users to establish a movement network. In contrast to individual trajectories, mining public movement can improve our understanding of how events are spread over time, general traffic conditions in different areas, etc.

Often spatial data is provided as a distinct point set in the LBSN domain. Graphs, in which the relationships between points can be represented as edges, are useful for representing and processing such point data. A *neighbourhood graph* is one where the edges are inferred based on the relationships defined on the point set provided for analysis. Distance-based connected neighborhood graphs are useful for inferring and processing movement networks between a set of distinct points [4]. Delaunay Triangulation (*DT*), Minimum Spanning Tree (*MST*), Gabriel Graph (*GG*) and Relative Neighbourhood Graph (*RNG*) are existing well known distance based connected neighbourhood graphs which are widely used to infer and process movement network between point data [4]. These graph approaches are *static* as they produce a unique movement network, represented as a *skeleton*, for any given point set. As an example, Figure 1(a) shows a location set collected from Twitter relating to a music festival. Figures 1(b,d,i,j) represent *DT*, *GG*, *MST* and *RNG* skeletons, respectively. Each neighbourhood skeleton is obtained by connecting the endpoints of the inferred edges using straight line segments which is referred to as the geometric realization of a neighbourhood graph and shows the geometric "shape" of the point set.

As opposed to the static graph approaches above, variable graph approaches produce a spectrum of possible skeletons, usually based on a single parameter. Hence variable graphs are more versatile [9]. The Shortest Path Graph (*SPG(t)*) [5] is such a variable graph, proposed with the idea of inferring edges between a point set such that the shortest path taken over the inferred edges will roughly align with the shortest path taken over the imprecise region represented by the point set. Characteristics of the inferred edge set varies based on a single user provided parameter $t \geq 1$. *SPG(t)* converges to a *MST* of the given point set as $t \rightarrow \infty$. We have found that the structure imposed by *SPG(t)*, when inferring edges using its distance based global criteria, is useful for some use cases, such as delineating imprecise regions [5] and overlapping set visualization [12], but is over-burdensome or unnecessary in other use cases such as movement analysis. Figures 1(c,e,g) represent skeletons generated using *SPG(t)* with $t = 2, 3, 4$, respectively.

To address the deficiency of existing approaches when considering the movement corridor use case, in this paper we propose a new type of variable graph, which we refer to as the Stepping Stone Graph (*SSG(d)*) with parameter $d \geq 1$. Briefly, similarly to the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGSPATIAL '18, November 6–9, 2018, Seattle, WA, USA

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-5889-7/18/11...\$15.00

<https://doi.org/10.1145/3274895.3274913>

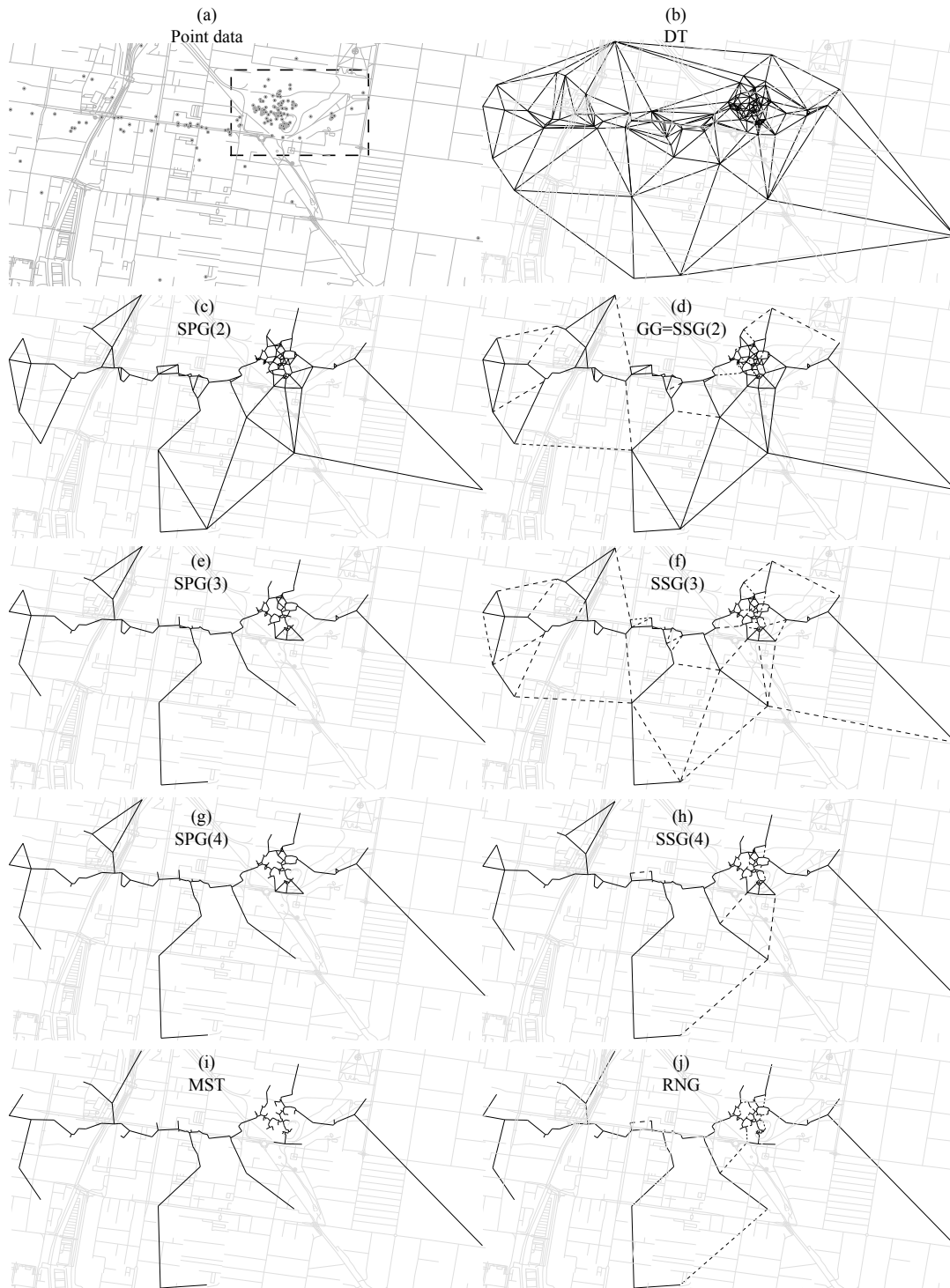


Figure 1: A point set (a), with a rectangle in dashed lines indicating the location where a music festival was held, and a set of connected neighbourhood skeletons (b,...,j). Dashed lines in (d,f,h,j) indicate additional edges inferred by our proposed Stepping Stone Graph compared to the Shortest Path Graph (c,e,g,i).

fact that $SPG(t)$ converges to MST , our $SSG(d)$ converges to RNG as $d \rightarrow \infty$. Thus, generating the RNG . Note that, RNG provides a structure more similar to human perception of a point set than the MST [18], which is advantageous. Therefore the spectrum of skeletons inferred by our $SSG(d)$ are likely to be more perceptually relevant compared to $SPG(t)$. We also show in our experiments that $SSG(d)$ is easier and faster to build and give better results in processing movement analysis related queries. This is because as the name suggests $SSG(d)$ focusses on where people have been and can step on for movement rather than any global characteristic.

To achieve this, $SSG(d)$ infers edges between a point set using a distance based local criterion based on the inherent properties of movement networks. $SPG(t)$ on the other hand create connections without enough consideration whether these connections can lead to a realistic step between two "stepping stones" on an area. Thus, we assume that social network data gives us partial data per individual user in terms of its path but with a good picture of where people could be in an event in a city. Intuitively, for all endpoint pairs, the value of d indicates the preference of inferring a longer alternative path with less distance between all point pairs on the path compared to the direct distance between the endpoint pair. This is useful when we have a very dense point set to cull some connections. Similarly to $SPG(t)$, as d increases, the number of edges in $SSG(d)$ monotonically decreases and therefore the path length between any two non-adjacent points in the skeleton monotonically increases. In Figure 1, figure pairs (c,d), (e,f), (g,h), and (i,j) depict $SPG(t)$ and $SSG(d)$ with $t = d = 2, 3, 4, \infty$, respectively. For each $d = t$ pair, additional edges inferred by $SSG(d)$ which are not inferred by $SPG(t)$ are highlighted using dashed lines, shown in Figures 1(d,f,h,j). It is important to note that GG is special case of $SSG(d)$ when $d = 2$.

In this paper, we use publicly available LBSN data to evaluate the utility of $SPG(t)$ and $SSG(d)$ for inferring a movement networks. We show that $SSG(d)$ performs better at applications such as popular path identification. Also given a DT , $SSG(d)$ can infer a planar edge embedding in $O(n)$ time where n is the number of points, whereas $SPG(t)$ takes $O(n^2 \log n)$. From our experiments, it is evident that $SSG(d)$ has a lower and more stable spanning ratio [1] compared to $SPG(t)$. Thus, $SSG(d)$ is more effective for inferring and processing the movement network compared to $SPG(t)$. We performed experiments on the analysis of movement networks for locations influenced by an event, and movement networks between localities posted within a city. In both experiments $SSG(d)$, for the case of $d = t$, inferred paths more similar to the real world paths, compared to $SPG(t)$. Our graph is also faster to compute. It also generalizes the RNG .

2 RELATED WORK

2.1 Spatial Analysis of LBSN Data

Our work focuses on visualizing the possible movement network using variable neighbourhood graphs. Following are some research that uses LBSN data to analyse interesting events to which the proposed skeleton can be added to visualize potential user flow patterns. MacEachren et al. [11] present results of a survey conducted on potential usage of social media in crisis management, with the participation of crisis management professionals. In order to cater

to the requirements gathered from the survey, authors propose a prototype system named SensePlace2. SensePlace2 allows the analyst to query the social media stream and visualize retrieved social media posts, a tag cloud and post distribution over a map. Chae et al. [3] present another visual analytic system for public behaviour analysis using social media data. This system features a map which allows analysts to add indicators to identify points of interest in the analysed areas and movements of natural disasters. Initially, they have tried to reveal public movement flows during interesting events. But, they were unable to find meaningful flows due to the movement randomness and the visual clutter of the flows. In both above systems, to provide a summarized view of post distribution, authors have noted the usefulness of geospatial heatmaps. Moreover, colouring overlay of the heatmap is computed using statistical methods.

2.2 Neighborhood Graphs

Neighbourhood Graphs infer edges between points based on some criteria. In neighbourhood graphs, edge inferring criteria can be defined per point, per point pair, or per all points in the sample. Neighborhood graphs that infer edges based on the absence of other vertices inside a region surrounding the endpoints of the edge are referred as Empty Region Graphs (ERG) [2]. When using an empty region as the evaluation criteria for edge inferring, it is important to note whether the boundary of the evaluation region is considered to be empty or not. If the boundary is considered to be empty then the evaluation region is referred to as a closed region and as an open region otherwise. It is a common practice to represent the boundaries of open and closed regions by dashed and solid lines, respectively when graphically representing them.

Both GG [7] and RNG [18] are static ERG that are useful for analysing the shape of a point set. GG is originally proposed as a tool for geographic variation analysis. Authors emphasize that the connectedness and planarity (non-existence of overlapping edges in skeleton) are crucial to make inferred edges intuitive for geographic data. As the empty region for GG , authors use the simple definition closed circle where inferring edge between a point pair is a diameter of the circle. RNG , which is introduced as a mechanism that can infer a structure close to human perception of a point set, uses open lune as the empty neighbourhood. Closed circle and open lune neighbourhoods for a vertex pair p, q , are shown in Figure 2 (a) and (b), respectively. In [2], closed circle and open lune are proved as the tight regions for planar and connected edge inferring, respectively. Note that $RNG \subseteq GG$. As later shown $SSG(2) \equiv GG$ and $SSG(\infty) \equiv RNG$.

β -Skeletons [9] are proposed to analyse shape of a point set and presented as a framework for the neighbourhood graphs which infer edges based on the circle based empty regions. As opposed to distance based definition of $SSG(d)$, β -Skeletons have a geometry based definition. Thus, $SSG(d)$ is more robust compared to β -Skeletons as later shown when analysing properties of $SSG(d)$. Authors present two categories of neighbourhoods. The lune based neighbourhoods are defined as an intersection of two circles, and the circle based neighbourhoods are defined as a union of two circles. With the lune based definition, GG is $\beta = 1$ closed neighbourhood skeleton and RNG is $\beta = 2$ open neighbourhood skeleton.

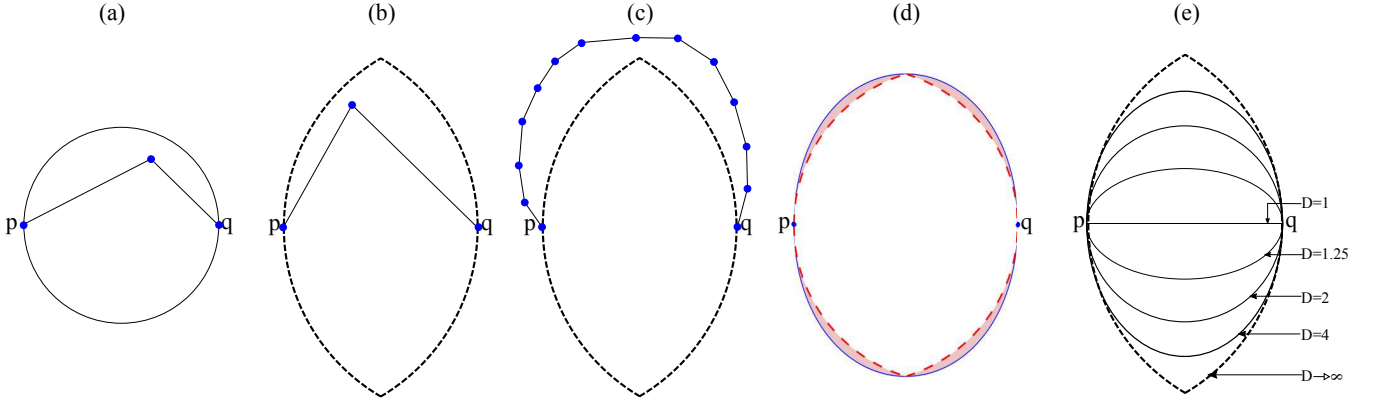


Figure 2: p and q are two points in space. Examples of (a) GG (b) RNG (c) $SPG(2)$ of point sets with (a) closed circle (b) and (c) open lune neighbourhoods (d) β Neighbourhood (at $\beta = 1.41421$) with dashed line on top of $DN(4)$ and (e) Graphical representation of $DN(pq, d)$

$SPG(t)$ is first proposed by Mark de Berg et al, as the base skeleton of a delineating mechanism to identify the boundary and cavities within an imprecise region, given as a positive locality sample [5]. When the weight of an edge is defined as its Euclidean length raised to the power of the parameter $t \geq 1$, the edge is present in $SPG(t)$ if and only if it is the least weight path between its endpoints. Authors show using empirical results that $SPG(2)$ is better for delineating an imprecise region, compared to both Kernel Density Estimation (KDE) and GG . $SPG(2)$ performs better than GG and β -Skeletons for delineating imprecise regions, because of its global criteria. Presenting $SPG(t)$ based delineating mechanism, the authors highlight that the connectedness of the resulting structure, created for analysis is a desirable property when processing an imprecise region provided as a positive locality sample [5]. Since open lune is the upper bound of the empty regions to ensure a connected skeleton, $SPG(2)$ should be compared against RNG . In Figure 2 (c), $SPG(2)$ does not contain an edge between p and q . However, for the same situation RNG infers an edge. Therefore, we can say that the RNG provides more alternative paths compared to $SPG(2)$ when inferring the movement network among the point set in Figure 2 (c). In the discussion, authors note that quality of result generated using proposed delineating method depends heavily on the parameter selection of outlier filtering. Later $SPG(t)$ skeletons with varying t are used as base structure for overlapping set visualization [12].

Like $SPG(t)$ that converges to MST as $t \rightarrow \infty$, $SSG(d)$ converges to RNG as $d \rightarrow \infty$. $SSG(d)$ infers a graph closer to human perception of the point set compared to $SPG(t)$ [18]. A key disadvantage of $SPG(t)$ is that using a global criterion in its inferring algorithm results in undesirable running times such as $O(n^2 \log n)$ to infer planar edges. Thus, generating $SPG(t)$ with varying t values is a computationally intensive task. In contrast, our proposed $SSG(d)$ utilizes a local evaluation criterion, which is inferred in $O(n)$ time given the DT for planar edge embedding.

3 STEPPING STONE GRAPH

We consider all location points provided for the analysis as stepping stones that need to be linked in a traversable manner. So we focus

on connecting these stepping stones such that a travelling entity can find a path between two locations.

3.1 Definitions

We construct $SSG(d)$ in the form of an undirected graph $G(V, E)$ where $V \subseteq \mathbb{R}^2$ represents a given point set and E represents inferred edges between the points. An edge between two endpoints $p, q \in V$ is represented as $pq \in E$. Length l_{pq} represents the Euclidean distance between two points.

First, we define the *diversion neighbourhood* in Euclidean space, which is the main construct necessary for defining $SSG(d)$, an area between two points which varies based on a single parameter $d \geq 1$.

Definition 3.1 (Diversion Neighbourhood). For $p, q \in \mathbb{R}^2$, the diversion neighbourhood of pq at $d \in \mathbb{R} : d \geq 1$, denoted $DN(pq, d)$ or simply $DN(d)$, is defined as the region:

$$DN(pq, d) = \left\{ z \in \mathbb{R}^2 : l_{pz}^d + l_{zq}^d \leq l_{pq}^d \right\}. \quad (1)$$

A graphical representation of $DN(pq, d)$ for varying d is shown in Figure 2(e). As the figure shows, $DN(d)$ enlarges from a straight line connecting the endpoints to the open lune neighbourhood between the endpoints. Note that we use $\leq$ instead of $<$ in Equation 1, making $DN(d)$ a closed region. Next, we define the Stepping Stone Graph using $DN(d)$.

Definition 3.2 (Stepping Stone Graph). For $V \subseteq \mathbb{R}^2$, the Stepping Stone Graph of V at $d \in \mathbb{R} : d \geq 1$, denoted $SSG(V, d)$ or simply $SSG(d)$, is defined as an undirected graph with points V , such that for each point pair $p, q \in V$:

$$pq \text{ is an edge of } SSG(V, d) \text{ iff } DN(pq, d) \cap V \setminus \{p, q\} = \emptyset.$$

Intuitively, if $DN(d)$ for a given d and endpoint pair contains no other points then $SSG(d)$ will have an edge between that endpoint pair.

Since $DN(d)$ becomes an open lune as $d \rightarrow \infty$, $SSG(d)$ is a connected graph, due to theorem 25 in [2]. Spectrum of skeletons inferred by $SSG(d)$ for the location set in Figure 1(a) is shown in Figure 1(d,f,h,j). The diversion neighbourhood is effectively a local criterion when selecting edges in $SSG(d)$, as opposed to the global

criterion in $SPG(t)$, and therefore in some cases $SSG(d)$ provides more alternative paths between a point set compared to $SPG(t)$, and $SSG(d)$ never provides less alternative paths than $SPG(t)$.

By definition of $DN(d)$, as d increases the travel distances over the shortest paths taken from $SSG(d)$ between all endpoint pairs monotonically increase. At the same time, the number of points on the shortest paths monotonically increases. Therefore, as d increases, the average distance to be travelled between a point pair over the shortest path taken from $SSG(d)$ monotonically decreases. Hence, $SSG(d)$ furnishes alternative paths where, for a given endpoint pair, each of the path edges in the alternative path is always shorter than the direct path between the endpoint pair.

3.2 Stepping Stone Graph Properties

As d increases, the area covered by $DN(pq, d)$ increases symmetrically around both the perpendicular bisector and the direct connecting line of pq . This leads to $SSG(d')$ being more restrictive than $SSG(d)$ for $d' > d$. Therefore, as d increases, by definition the number of edges in $SSG(d)$ monotonically decreases. Hence, the following theorem results (proofs omitted for space considerations).

THEOREM 3.3. For $1 \leq d \leq d'$, $SSG(d') \subseteq SSG(d)$

By definition of $DN(d)$ and by the Thales' Theorem, $DN(pq, 2)$ becomes a closed circumcircle of pq and as $d \rightarrow \infty$, $DN(d)$ becomes the open lune. Therefore, the following two theorems result:

THEOREM 3.4. $SSG(2) \equiv GG$

THEOREM 3.5. As $d \rightarrow \infty$, $SSG(d) \rightarrow RNG$

By Theorem 3.5 we refer to $\lim_{d \rightarrow \infty} SSG(d)$ as $SSG(\infty) \equiv RNG$ in later sections. The following lemmas are trivial due to Theorems 24 and 25 of the Empty Region Graph study [2]:

LEMMA 3.6. For $d \geq 2$, $SSG(d)$ is planar.

LEMMA 3.7. For all $d \geq 1$, $SSG(d)$ is connected.

By combining lemmas 3.6 and 3.7, the range of d where $SSG(d)$ is both connected and planar can be deduced.

LEMMA 3.8. For $d \geq 2$, $SSG(d)$ is both planar and connected.

Now we compare SSG to SPG .

THEOREM 3.9. For $d \leq t$, $SPG(t) \subseteq SSG(d)$

PROOF. Define the *edge weight* of pq with respect to t as l_{pq}^t , for some $t \geq 1$. Assume that for all $z \in V \setminus \{p, q\}$, $l_{pz}^t + l_{zq}^t > l_{pq}^t$. In this case, pq is an edge in $SPG(t)$. Now we show that for $d \leq t$, pq is also an edge in $SSG(d)$. Let us write $d = t\epsilon$ where $\frac{1}{t} \leq \epsilon \leq 1$. Then we need to show that:

$$\begin{aligned} \frac{l_{pz}^{t\epsilon} + l_{zq}^{t\epsilon}}{l_{pq}^{t\epsilon}} &> 1 \\ \left(\frac{l_{pz}}{l_{pq}}\right)^{t\epsilon} + \left(\frac{l_{zq}}{l_{pq}}\right)^{t\epsilon} &> 1 \\ \left(\left(\frac{l_{pz}}{l_{pq}}\right)^t + \left(\frac{l_{zq}}{l_{pq}}\right)^t\right)^{\frac{1}{\epsilon}} &> 1 \end{aligned} \quad (2)$$

Since the function $x \mapsto x^\beta$ is subadditive for $\beta \geq 1$ then:

$$\left(\left(\frac{l_{pz}}{l_{pq}}\right)^t + \left(\frac{l_{zq}}{l_{pq}}\right)^t\right)^{\frac{1}{\epsilon}} \geq \left(\frac{l_{pz}}{l_{pq}}\right)^t + \left(\frac{l_{zq}}{l_{pq}}\right)^t.$$

We know the right hand side is greater than 1 due to our initial assumption and therefore Eq. 2 is true. Therefore pq is also an edge in $SSG(d)$ and this completes the proof. $\square$

Now we consider the relationship between $SSG(d)$ and β -Skeletons. The empty regions used for both the Lune based β -Skeleton [9] and $SSG(d)$ are symmetric around the perpendicular bisector of the straight line connecting point pair. Using this similarity, we analysed how these neighbourhoods relate to each other for $d \geq 2$. As depicted on Figure 2(d), we observed that the β -Skeleton touches $DN(d)$ only on four points in the range of $\beta = (1, 2)$. Therefore, we can say that $DN(d)$ is more robust compared to the lune based neighbourhood of β -Skeleton in the range $\beta = (1, 2)$. We used the fact that Euclidean distances between these four points should be equal, to analyse the relationship between β and d . As a result of this analysis, we discovered following theorem.

THEOREM 3.10. For all $d \geq 2 \wedge \beta \leq 2^{(1-2/d)}$,
 $SSG(d) \subseteq \text{lune based } \beta\text{-skeleton}$

Next we consider how $SSG(d)$ relate to DT .

THEOREM 3.11. For $d \geq 2$, $SSG(d) \subseteq DT$

PROOF. By theorems 3.5 and 3.4, $SSG(\infty) \subseteq SSG(2) \subseteq DT$. Let's assume DT excludes an edge which should be present in $SSG(d)$ for $d \geq 2$. For DT to exclude an edge, endpoints of that edge are not Voronoi neighbors. An edge between a point pair which are not Voronoi neighbours will not create a planar graph. But, by Lemma 3.6 excluded edge cannot be in $SSG(d)$ for $d \geq 2$. Thus, our initial assumption is wrong. Hence by proof of contradiction we establish that for $d \geq 2$, $SSG(d) \subseteq DT$. $\square$

3.3 Algorithms

This section presents the algorithms to compute $SSG(d)$ and how to compute paths based on $SSG(d)$. Due to Theorem 3.11, we can use the complete graph (CG) and the DT graph as starting graphs for extracting $SSG(d)$ for the ranges $2 > d \geq 1$ and $d \geq 2$, respectively. To make the extraction process more efficient, we label the edges spanned by these graph structures with the minimum value of d necessary for the edge to be excluded from $SSG(d)$. We define the starting graph structure with edges labelled with these minimum d values as the d -spectrum.

Definition 3.12 (d -spectrum). For an edge pq , the smallest real number $d \geq 1$ such that $DN(d) \cap V \setminus \{p, q\} \neq \emptyset$ is called the d -value for pq . The set of all edges spanned by V , each labelled by its d -value is called the d -spectrum of V , denoted as $d\text{-spectrum}(V)$ or simply $d\text{-spectrum}$.

For $d \geq 1$, a naive algorithm can be formulated by evaluating all $z \in V \setminus \{p, q\}$ against each point pair $p, q \in V$ and comparing to find the d -value of pq . All edges which have other points on them will be discarded as they will never be present in $SSG(d)$. For the

cases where an open lune neighbourhood is empty for an endpoint pair, the edge is labelled with ∞ as its d -value. All remaining edges will be labelled with d -values in the range of $1 \leq d < \infty$. For a V with n points, the number of edges that connect each point pair is in $O(n^2)$. As each edge is compared against the other $(n-2)$ points, the time complexity of this naive algorithm is $\Theta(n^3)$.

3.3.1 Planar d -Spectrum(V). For $d \geq 2$, DT can be used as a starting point due to Theorem 3.11. As a naive algorithm, for each point pair $p, q \in V$, where there is an edge between those points in DT , all $z \in V \setminus \{p, q\}$ can be evaluated to find the d -value of edge pq . For a V with n number of points, number of edges in DT is $O(n)$. As each edge is compared against other $(n-2)$ points, the time complexity of this algorithm is $\Theta(n^2)$.

A more efficient algorithm to obtain the d -Spectrum of edges spanned by $DT(V)$ is shown in Algorithm 1. In this approach, we are sweeping the triangles in $DT(V)$ that are intersecting with the relative neighbourhood of each $p, q \in V : pq \in DT(V)$. The neighbouring triangles of $pq \in DT(V)$ are represented by the set $\Lambda(pq)$. $\Omega(T)$ denote the circumscribed circle of a triangle T . $I(DN(d))$ denote the interior of $DN(D)$, while $B(DN(d))$ denote the boundary of $DN(D)$. We start the process for each $pq \in DT(V)$ with one neighbouring triangle of pq (Line 7). And we continue sweeping the triangles intersecting with the line segment connecting the center point of pq and $Zmax$ which is the furthest away point in $DN(d)$ from pq . Given the Delaunay triangulation, this algorithm runs in $O(n)$ time as it only examines the points in the relative neighbourhood of each edge.

When presenting the time complexities of proposed algorithms above, we assume that the time taken to solve for the d -value of an edge is $O(1)$, i.e. that we ask for constant precision. Note that all of the above algorithms are readily parallelizable as there is no race condition between separate edge evaluations.

3.3.2 Computing $SSG(V, d)$. Once the d -spectrum of V is calculated for a given d range, extracting the edges of $SSG(d)$ is a straightforward process. As d -Spectrum indicates d -value of each edge, edges that have d -value greater than given d can be filtered out as the edge set of $SSG(d)$. This process will take $O(m)$ time complexity where m is the number of edges resulting from the method used to calculate the d -Spectrum. For $1 \leq d < 2$, m would be $O(n^2)$, and for $2 \leq d$, m would be $O(n)$.

3.3.3 Improving running time of $SPG(V, t)$. Using the d -spectrum, we can exclude some of the edges from $SPG(t)$, without calculating the alternative shortest path, to improve the running time of $SPG(t)$ extraction. By Theorem 3.9, the edges of the d -spectrum that have a d -value less than or equal to a given t value, will not be in the $SPG(t)$. This property can be used to skip edges from the shortest path calculation part of $SPG(t)$ extraction algorithm [12]. It should be noted that even though this method improves the running time of the $SPG(t)$ algorithm, it does not reduce the time complexity of the worst case.

3.3.4 Calculating paths using $SSG(d)$. After calculating $SSG(d)$, Dijkstra's algorithm can be used to find the shortest paths between any two endpoints. As $SSG(d)$ contains more alternative paths compared to $SPG(t)$, paths taken over $SSG(d)$ will be shorter and more similar to real world paths.

Algorithm 1: Planar d -Spectrum(V)

Input: V - Filtered locality set
Output: Every edge $pq \in DT(V)$ marked with d -value

```

1  $DT \leftarrow$  create Delaunay Triangulation of  $V$ ;
2 foreach (Edge  $pq : pq \in DT$ ) do
3    $d \leftarrow \infty$ ;
4    $ME \leftarrow pq$ ;
5    $C \leftarrow$  Center of  $pq$ ;
6   foreach ( $T \in \Lambda(pq)$ ) do
7      $MT \leftarrow T$ ;
8      $Zmax \leftarrow Zmax \in DN(d) : \max[l_{CZmax}]$ ;
9     while ( $MT \neq \emptyset$ ) do
10      if ( $Zmax \in \Omega(MT)$ ) then
11        break;
12      end
13       $MV \leftarrow$  Vertex  $Z \in MT : z \notin ME$ ;
14      if ( $MV \in I(DN(d))$ ) then
15         $d \leftarrow d : MV \in B(DN(d))$ ;
16        update  $Zmax$ ;
17      end
18       $ME \leftarrow \exists \text{edge } E \in MT \setminus ME : ME \cap CZmax \neq \emptyset$ ;
19      if ( $ME = \emptyset$ ) then
20        break;
21      end
22       $MT \leftarrow T1 \in \Lambda(ME) : T1 \neq MT$ ;
23    end
24  end
25  set  $d$  as  $d$ -Value of  $pq$ ;
26 end

```

3.4 Applications

As shown in the experiments section later, $SSG(d)$ skeleton itself can be used to represent and process the travel network between a set of related localities. In this scenario, inferred edges represent the space occupied by an entity when moving between locations. Following are some application scenarios that can benefit from $SSG(d)$.

3.4.1 Nearest Neighbour Queries. Created graph structure can be used to search nearest interesting locations from the current location and to get the path to travel in order to get there. We propose to use breadth first search starting from the query location and traverse the graph until a required interesting point is found.

Definition 3.13 (Nearest neighbour (NN)). A location (L) from an interesting location set (IL) such that the shortest path distance from current location to that location is smaller compared to the shortest paths to all other locations,

$$NN = \{\exists L \in IL : \forall x \in IL, \text{length}(\text{path}(L)) \leq \text{length}(\text{path}(x))\}$$

Similarly we can calculate k -nearest neighbours. Instead of stopping breadth first search when first interesting point is found, it can be continued until k interesting points are found. As for the edge weights, we can use weights calculated in the section 3.4.3

according to the usage of edges. This will make sure that the most popular path to the nearest neighbour will be found.

3.4.2 Refined Movement Corridors. Once $SSG(d)$ is created using posted localities, user trajectories can be used to refine the created travel network. For each consecutive location pair in user trajectories, the shortest path is determined using $SSG(d)$. With each $SSG(d)$ edge the number of trajectories passed through that edge is recorded. We represent movement corridors in the travel network based on edges that contain trajectory count higher than a given threshold.

Definition 3.14 (Usage counter). When $path$ is a sequence of edges traversed by a trajectory trace, for all $pq \in E$, Usage Counter of pq (denoted $UC(pq)$), is defined as the trajectory count,

$$UC(pq) = |\{path : pq \in path\}|$$

3.4.3 Most Popular Paths. After calculating usage counters of $SSG(d)$ edges, they can be used to find the most popular path between locations. To find most popular path, we calculate edge weight to reflect popularity of the edge and use shortest path algorithm to calculate the paths. To ensure edges with more usage have lower weights, we divide length of the edge by usage counter of that edge. For edges with no usage, edge length multiplied by a constant greater than 1 is used as the edge weight. After calculating edge weights in this manner, the Dijkstra's shortest path algorithm is used to find most popular path between two locations.

Definition 3.15 (edge weight). For all $pq \in E$, weight of pq is defined as,

$$weight(pq) = \begin{cases} l_{pq}/UC(pq), & \text{if } 0 < UC(pq). \\ l_{pq} \times C : 1 < C, & \text{if } UC(pq) = 0. \end{cases}$$

3.4.4 Tour recommendation. Above most popular path calculation mechanism can be used to calculate the most popular paths between tourist attractions. Given tourist attractions and user trajectories within a city, first we calculate $SSG(d)$ and edge weights based on usage. Then, we apply most popular path calculation between any two selected attractions to find the path popular among tourists to travel between the selected attractions. Sequence of tourist attractions that appear on the calculated most popular path is reported as a recommended tour along with the path.

4 EXPERIMENTS

We perform experiments to see the effectiveness of $SSG(d)$ against $SPG(t)$ on inferring movement networks.

4.1 Data Sets

We conducted experiments on two datasets. The first data set consists of Geo-located posts collected from Twitter. A post set collected from 06th March to 23rd April 2012, within a bounding box over the countries Australia and New Zealand is used as the data set for our analysis. It contains 724651 LBSN posts authored by 36639 users. For our analysis a LBSN post is defined as a tuple containing four elements - userID to identify authoring user, voluntarily generated textual content, timestamp at which and locality where the post was authored.

We used Yahoo! Flickr Creative Commons 100M (YFCC100M) dataset [17] as second dataset. It contains meta data such as user

information, timestamp and location where photo was taken of 100 million photos and videos shared on site Flickr. Only the entries with point geo-locations were used for our experiments. For refined movement corridor and tour recommendation experiments localities in London, England are filtered out from the YFCC100M data set.

4.2 Implementation

To visualize the inferred neighbourhood skeletons, a visualization tool was implemented utilizing GeoTools¹ Java libraries. All the skeleton visualizations presented in all figures were generated using this tool. Both $SSG(d)$ and $SPG(t)$ algorithms were implemented using Java 8. The data set was stored in a MongoDB database and unique locality set relating to a given event was queried from it as V . We used travel network filtered from OpenStreetMap (OSM)² to get a better understanding of the area being analysed. When filtering travel network of an area from OSM data we took foot paths, cycle paths, roads and rail roads.

To infer planar $SSG(d)$, firstly, DT was created using Sweep-Hull [16] algorithm in $O(n \log n)$ time. Then, planar d -Spectrum is calculated using Algorithm 1. Finally, $SSG(d)$ is extracted from d -Spectrum created using DT . We used numerical analysis to calculate d -value of an edge. More specifically, a Java method was implemented to perform Secant method³ to approximate the d -Value. For extracting planar $SPG(t)$, we used Algorithm 2 presented in [12].⁴ For varying $SSG(d)$ and $SPG(t)$ skeleton extractions, created DT was reused.

4.3 Event Analysis

An event can be viewed as an extraordinary spatiotemporal phenomenon that motivates LBSN users to generate posts within the spatial and temporal bounds of the event with content identifying the event. Following this perspective, we define an event as a tuple containing three elements - a spatial bound, a temporal bound and a description of content identifying the event. As the Twitter data we are using has textual content, we use a regular expression as the description of the event.

To discuss the properties of $SSG(d)$ and $SPG(t)$ skeletons we selected locality set relating to EasterFest 2012⁵ - an annual music festival held in Toowoomba, Australia from the Twitter data set. For temporal bound of EasterFest, we took the time period between 04th and 10th of April 2012. As for the spatial bound of EasterFest, we considered a bounding box (indicated by dashed lines in Figure 1 (a)) over the Queens Park, Toowoomba and surrounding main streets where the event was held. We consider all users who have posted with "#EasterFest" hashtag within spatial and temporal bounds of the event as the user set influenced by the event.

The locality filtering process can be seen as a three step process. First, the users who have authored content matching the description of the event within the event's temporal and spatial bounds are

¹<http://www.geotools.org/>

²<https://www.openstreetmap.org/>

³https://en.wikipedia.org/wiki/Secant_method

⁴ However, we noticed that $\geq$ in line 5 violates the edge selection criteria proposed as equality allows paths with similar weights to exists between an edge pair. Therefore we removed equality in line 5 when creating our implementation.

⁵<http://www.easterfest.com.au>

filtered. Second, the LBSN trajectories within the event's temporal bounds generated by the users filtered in the first step are taken as the LBSN posts set influenced by the event. Third, unique locality set from posts filtered in second step are taken as the V for variable skeleton generation. Relating to Easterfest, 183 unique points were filtered. Figure 1 (a) depicts the filtered locality set along with the travel network of the area analysed. We used the rectangle marked with the dashed lines as spatial bound of the event. This rectangle encloses the event venue (Queens Park, Toowoomba) and the main streets around it as described above.

4.4 Comparison of Two Graphs

As experiments for filtered locality set (V) relating to EasterFest, we generated varying skeletons with $SSG(d)$ and $SPG(t)$ at $d = t$. After that, generated skeletons are analysed to compare the changing characteristics of the skeletons as parameters vary. We used locality set relating to an event because all users participating in the event are there for a common reason and exhibit a similar movement pattern. To further understand how graph structures behaved, the number of edges and spanning ratio [1] of the graph structure were plotted with varying configuration parameters (Figure 3).

Figure 1 (a) and (b) present the locality set used as the V and the DT skeleton. Three skeletons on the middle left c, e and g represent the $SPG(t)$ with t set to 2, 3 and 4, respectively. While three skeletons on the middle right d, f and h represent $SSG(d)$ with d set to 2, 3 and 4, respectively.

At the first glance, we can see that $SPG(t)$ is a subgraph of $SSG(d)$ when $d \leq t$ (Theorem 3.9). Also, in both $SSG(d)$ and $SPG(t)$, edge counts are monotonically reducing (Figure 3 (a)). Further more, number of edges in $SPG(t)$ is always smaller than that of $SSG(d)$. This is to be expected as $SPG(t)$ extraction uses a global criteria compared to $SSG(d)$ and infer less edges. Figure 3 (b) shows the variation of spanning ratio as configuration parameter varies to demonstrate how the shortest path distances between locality pairs change. Spanning ratio of a graph indicates the maximum ratio between the shortest path distance over the graph and direct distance between any point pair. Therefore, graphs with low spanning ratio are preferred to represent movement networks [1]. Since $SSG(d)$ has a low and stable spanning ratio compared to $SPG(t)$, $SSG(d)$ is suitable for movement analysis. Furthermore, when $SPG(t)$ and $SSG(d)$ are created with the same number of edges, $SPG(t)$ tends to have a higher spanning ratio compared to $SSG(d)$.

Next we conduct an experiment to see how similar are paths from $SSG(d)$ and $SPG(t)$ when compare to real world paths taken from OpenStreetMaps. We randomly selected 10 point pairs from filtered point set and calculated path distances between them using OpenStreetMap paths, $SPG(3)$ and $SSG(3)$. We selected 3 as the configuration parameter for both $SSG(d)$ and $SPG(t)$ as it provides a general movement network between localities. For 9 point pairs, paths taken using $SSG(3)$ were different by 20% or less by length compared to real world path taken from OpenStreetMap. However, for paths taken using $SPG(3)$ only 6 pairs were in the same bracket. Therefore, we can see that paths inferred using $SSG(d)$ are more similar to real world paths compared to paths inferred using $SPG(t)$.

We also calculated nearest neighbour queries using created graph structures. As we used breadth first search for querying nearest

neighbours, observed results were similar to results of the shortest path calculations above. Paths calculated to nearest neighbours using $SSG(d)$ were more similar to real world paths compared to paths calculated using $SPG(t)$.

When analysing the spectrum of graph skeletons presented in Figure 1, we can see that as configuration value increases, inferred $SSG(d)$ skeleton emphasizes more on tight locality clusters, by thinning longer edges. Relating to the selected event by analysing tweets, it can be seen that filtered localities are densely centered towards the Points of Interest (POIs) such as event location and main streets in the area. This phenomenon makes it easier to visualize POIs clearer, along with the possible movement network between POIs. Furthermore, when comparing inferred skeletons, against the road maps available via public sources, it is observed that inferred edges of both $SSG(d)$ and $SPG(t)$ align better with road network as locality density increases in roads. But, as locality density varies, $SSG(d)$ skeleton highlights more alternative paths compared to $SPG(t)$ with the same configuration value.

In order to generate varying visualizations, skeleton needs to be visualized with varying parameters manually. Time taken to calculate skeletons of $SSG(d)$ and $SPG(t)$ are shown in Figure 3 (c). Execution time for $SSG(d)$ calculation is less compared to $SPG(t)$ for all configuration values. To generate $SSG(d)$ skeleton of given point set, we need to calculate the d -spectrum once and can extract skeleton for desired d from it. Once d -spectrum is calculated $SSG(d)$ skeleton can be rendered in $O(m)$ time, where m is the number of edges spanned by V . Therefore, we can generate varying $SSG(d)$ skeletons efficiently. For $SPG(t)$, we have to execute iterative shortest path algorithm every time t changes, making analysis process inefficient. But, we can reduce this time using calculated d -Spectrum. However, this does not reduce the worst case time complexity of $SPG(t)$ algorithm.

4.5 Further Applications

4.5.1 Refined Movement Corridors. Refined movement corridors refer to edges of the graph that are used for movement. These edges are selected by aligning user trajectories along the graph edges using shortest path calculation. We analysed refined movement corridors relating to the trajectories filtered from YFCC100M dataset, around Thames river in London. To represent the travel networks, $SPG(3)$ and $SSG(3)$ were used. This data set was selected because it had a lot of tourist movement compared to the Twitter data set. After that trajectories are aligned along both graph skeletons, and all the edges with usage counter more than 5 are filtered as refined movement corridors. Intuitively if an edge is used by trajectories 5 times, that edge is selected as a refined movement corridor. Visualization created using filtered edges are shown in Figure 4. In both refined movement corridors represented by thick edges appear along the banks of the river and on bridges where movements happen. Edges created across river are filtered out as there is no movement happening. When all trajectories are aligned against the refined movement corridors calculated using $SSG(3)$ and $SPG(3)$, 14% more of trajectories appear on refined movement corridors calculated using $SSG(3)$ compared to $SPG(3)$. Hence refined movement corridors created using $SSG(d)$ are more similar

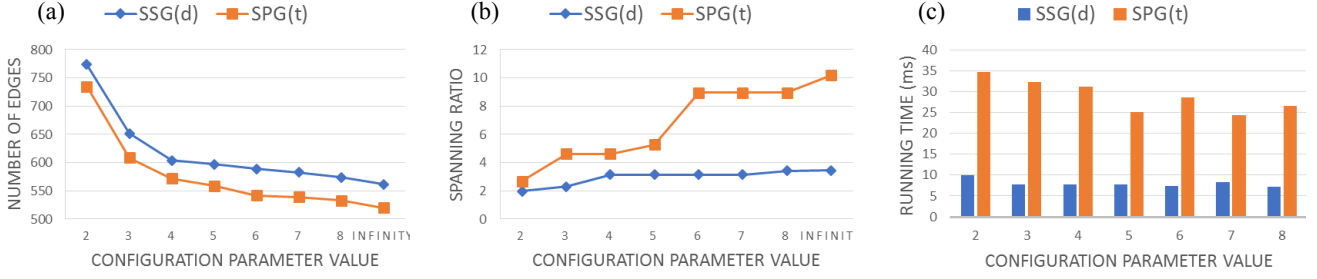


Figure 3: (a) number of edges (b) spanning ratios and (c) execution times of $SSG(d)$ and $SPG(t)$ when configuration parameters d and t vary

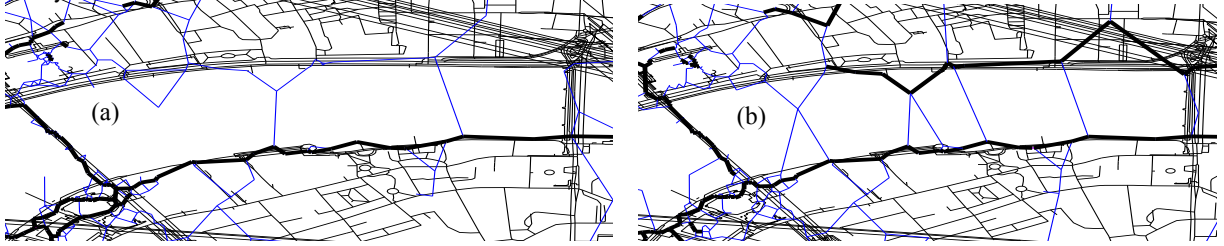


Figure 4: Thick lines indicate refined movement corridors extracted using (a) $SPG(3)$ and (b) $SSG(3)$

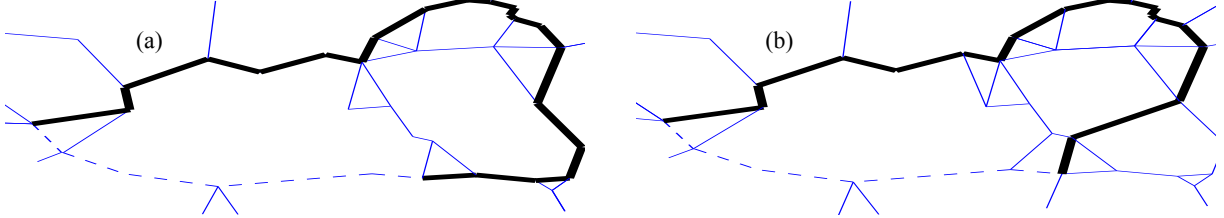


Figure 5: Paths calculated between two arbitrary locations using (a) $SPG(3)$ and (b) $SSG(3)$. Dashed line indicates shortest path and thick line indicates the most popular path. Note that the most popular path calculated with $SSG(3)$ is shorter than that of $SPG(3)$

to real world paths compared to refined movement corridors calculated using $SPG(t)$. As this method indicates most used part of the travel network, it can be used to generate preferred part of a travel network used for calculating preferred paths [8].

4.5.2 Most Popular Paths. After calculating refined movement corridors over an area, edge weights are set to exhibit the usage of that edge as shown in section 3.4.3. Most popular paths between two locations can be found by calculating the shortest path using above calculated edge weights. In order to experiment the effectiveness of the most popular path finding mechanism, we applied it to find a path around a junction in Melbourne Central Business District (CBD). Data for the experiment is taken from the Twitter data set. As twitter data set was restricted to the region Australia, it has a lot of movement information of Twitter users relating to their daily travels. One example most popular path is shown in Figure 5. $SPG(3)$ and $SSG(3)$ were used to represent travel networks, respectively. Calculated shortest paths are shown in broken line and most popular paths are shown in thick edges. As shown in the Figure 5 most popular path inferred using $SSG(3)$ is shorter compared to that of $SPG(3)$. In fact when most popular paths are calculated for all trajectories, most popular paths calculated using $SSG(3)$ were 18% shorter than most popular paths calculated using

Table 1: Comparison of tour recommendations calculated using $SPG(3)$ and $SSG(3)$

Graph	Recall	Precision	F ₁ -score
$SPG(3)$	0.699 ± 0.050	0.681 ± 0.060	0.676 ± 0.050
$SSG(3)$	0.777 ± 0.050	0.783 ± 0.060	0.774 ± 0.059

$SPG(3)$. In this particular case when selected most popular paths are compared with travel network taken from OpenStreetMaps, it is seen that there are roads that align with the most popular path whereas no actual road aligns with shortest paths.

4.5.3 Tour recommendation. Using YFCC100M data set we experimented effectiveness of tour recommendation in city of London. We followed the experimental set up followed in [10]. Geo-located photos posted over two months time period were used for experiments. All tourist travel sequences set apart by one hour time intervals are taken as query data set. For each tourist attraction sequence travelled by users, we calculated the most popular path between first and last tourist attractions they've been to and selected tourist attractions that coincide with the most popular path as recommended tour itinerary. For each real world travel sequence

(P_v), a recommended tour itinerary (P_r) was calculated using both $SPG(3)$ and $SSG(3)$. For tours recommended by both graphs tour recall ($T_R = |P_r \cap P_v|/|P_v|$), tour precision ($T_P = |P_r \cap P_v|/|P_r|$) and tour F₁-score ($T_{F1} = (2 \times P_r \times P_v)/(P_v \times P_r)$) [10] were calculated and presented in Table 1. As shown for all three criteria $SSG(3)$ evaluates higher values than $SPG(3)$. Thus, we can say that the inferences using $SSG(d)$ are more close to real world user travels than $SPG(t)$.

4.6 Configuration Value

We selected 3 as the configuration value to compare resulting graphs generated using $SSG(d)$ and $SPG(t)$. For configuration value 1 both $SSG(d)$ and $SPG(t)$ are same and do not cull edges to cater for point density. And for configuration parameter value 2, $SSG(d)$ become GG which is a well known static graph. Configuration parameter value 3 gives interesting results so we have used it. Configuration values greater than 3 give the similar results as the results generated with configuration parameter set to 3.

5 FUTURE DIRECTIONS

By analysing experimental results, we can observe that $SSG(d)$ is both effective and efficient for representing and processing the possible movement network. Therefore, proposed analysis method can be used in situations such as analysing movement patterns of animals where similar locality distributions are observed. In our experiment only the spatial and temporal aspects of the LBSN data is considered for analysis without geographic information of the area. Furthermore, the parameter selection for variable skeleton generation is considered as a manual process. Following future development can be highlighted based on our experiment.

- Autonomous detection of a suitable value for parameter d
- Investigation on information that can be visualized using other aspects such as sentimental aspect of LBSN data
- Development of colouring overlays based on created skeleton

Analysis of trajectories resulting from moving objects based on $SSG(d)$ can be listed as a promising future direction. Further more, $SSG(d)$ can be used for movement planning by filtering out edges longer than a given threshold [4]. With the introduction of $SSG(d)$ as a super graph of $SPG(t)$ when $d \leq t$, it is worth investigating how use cases of $SPG(t)$ can be further improved. Since $SSG(d)$ emphasizes on tight locality clusters as d increases, we may be able to break the connectivity property of the graph and identify clusters.

Since we are analysing geographic data we limited the $SSG(d)$ definition to 2-dimensional case with Euclidean distance. As $DN(d)$ definition is distance based, concept of $SSG(d)$ can be extended to higher dimensions. As planarity of extracted skeleton is observed to be a desired property, we only presented results with configurable value range of $d \geq 2$. But, there are use cases that can benefit from value range $d < 2$ [14]. Therefore, research on how the non-planar region of $SSG(d)$ behave is another interesting direction.

6 CONCLUSION

We analysed the suitability of variable connected neighbourhood skeletons to visualize the possible movement network of LBSN users influenced by events. We presented Stepping Stone Graph ($SSG(d)$) of a finite planar set. It is a connected graph that varies

depending on a single parameter d . We analysed how $SSG(d)$ relates to some well-known graph structures. And we presented algorithms to calculate $SSG(d)$. In addition, we presented how $SSG(d)$ can be used to improve running time of Shortest Path Graph ($SPG(t)$). We have empirically shown that $SSG(d)$ is both efficient and effective to visualize the possible movement network using LBSN data due to its distance based local evaluation criteria. Furthermore, we have shown the usefulness of $SSG(d)$ in few applications.

ACKNOWLEDGMENTS

This research is funded in part by the Defence Science and Technology Group, Edinburgh, South Australia, under contract MyIP:6104.

REFERENCES

- [1] Prosenjit Bose, Luc Devroye, William S. Evans, and David G. Kirkpatrick. 2006. On the Spanning Ratio of Gabriel Graphs and beta-Skeletons. *SIAM J. Discrete Math.* 20, 2 (2006), 412–427.
- [2] Jean Cardinal, Sébastien Collette, and Stefan Langerman. 2009. Empty region graphs. *Computational geometry* 42, 3 (2009), 183–195.
- [3] Junghoon Chae, Dennis Thom, Yun Jang, SungYe Kim, Thomas Ertl, and David S. Ebert. 2014. Public behavior response analysis in disaster events utilizing visual analytics of microblog data. *Computers & Graphics* 38 (2014), 51–60.
- [4] Jorge Cortés, Sonia Martínez, and Francesco Bullo. 2006. Robust rendezvous for mobile autonomous agents via proximity graphs in arbitrary dimensions. *IEEE Trans. Automat. Control* 51, 8 (2006), 1289–1298.
- [5] Mark de Berg, Wouter Meulemans, and Bettina Speckmann. 2011. Delineating imprecise regions via shortest-path graphs. In *19th ACM SIGSPATIAL International Symposium on Advances in Geographic Information Systems, ACM-GIS 2011, November 1–4, 2011, Chicago, IL, USA, Proceedings*, Isabel F. Cruz, Divyakant Agrawal, Christian S. Jensen, Eyal Ofek, and Egemen Tanin (Eds.). ACM, 271–280.
- [6] Matt Duckham, Marc J. van Kreveld, Ross Purves, Bettina Speckmann, Yaguang Tao, Kevin Verbeek, and Jo Wood. 2016. Modeling Checkpoint-Based Movement with the Earth Mover’s Distance. See [13], 225–239.
- [7] K Ruben Gabriel and Robert R Sokal. 1969. A new statistical approach to geographic variation analysis. *Systematic Biology* 18, 3 (1969), 259–278.
- [8] Erik G. Hoel, Shawn D. Newsam, Siva Ravada, Roberto Tamassia, and Goce Trajcevski (Eds.). 2017. *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, GIS 2017, Redondo Beach, CA, USA, November 7–10, 2017*. ACM.
- [9] David G Kirkpatrick and John D Radke. 1985. A framework for computational morphology. In *Machine Intelligence and Pattern Recognition*. Vol. 2. Elsevier, 217–248.
- [10] Kwan Hui Lim, Jeffrey Chan, Christopher Leckie, and Shanika Karunasekera. 2015. Personalized Tour Recommendation Based on User Interests and Points of Interest Visit Durations. In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25–31, 2015*, Qiang Yang and Michael Wooldridge (Eds.). AAAI Press, 1778–1784.
- [11] Alan M. MacEachren, Anuj R. Jaiswal, Anthony C. Robinson, Scott Pezanowski, Alexander Savelyev, Prasenjit Mitra, Xiao Zhang, and Justine I. Blanford. 2011. SensePlace2: GeoTwitter analytics support for situational awareness. In *2011 IEEE Conference on Visual Analytics Science and Technology, VAST 2011, Providence, Rhode Island, USA, October 23–28, 2011*. IEEE Computer Society, 181–190.
- [12] Wouter Meulemans, Nathalie Henry Riche, Bettina Speckmann, Basak Alper, and Tim Dwyer. 2013. KelpFusion: A Hybrid Set Visualization Technique. *IEEE transactions on visualization and computer graphics* 19, 11 (2013), 1846–1858.
- [13] Jennifer A. Miller, David O’Sullivan, and Nancy Wiegand (Eds.). 2016. *Geographic Information Science - 9th International Conference, GIScience 2016, Montreal, QC, Canada, September 27–30, 2016, Proceedings*. Lecture Notes in Computer Science, Vol. 9927. Springer.
- [14] Joon C. Park, Hayong Shin, and Byoung Kyu Choi. 2006. Elliptic Gabriel graph for finding neighbors in a point set and its application to normal vector estimation. *Computer-Aided Design* 38, 6 (2006), 619–626.
- [15] Dipto Sarkar, Renée Sieber, and Raja Sengupta. 2016. GIScience Considerations in Spatial Social Networks. See [13], 85–98.
- [16] David Sinclair. 2016. S-hull: a fast radial sweep-hull routine for Delaunay triangulation. *CoRR abs/1604.01428* (2016).
- [17] Bart Thomee, David A. Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. 2016. YFCC100M: the new data in multimedia research. *Commun. ACM* 59, 2 (2016), 64–73.
- [18] Godfried T Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern recognition* 12, 4 (1980), 261–268.